FAQ для судей

Английский оригинал: FAQ_FOR_JUDGES.md.

Пятнадцать вопросов, с которых судья, вероятно, начнёт, и короткий ответ на каждый. Два-три предложения, затем файл или команда, которые его подкрепляют. Полный список со ссылками на источники — docs/DEFENSE_FAQ.md; числа — в docs/METRICS.md и docs/REAL_CORPUS_ANALYSIS.md.

1. Почему не использовать Wireshark для разбора?

Wireshark — программа, а не библиотека, поэтому его использование означало бы вызов tshark и зависимость от внешнего бинарника. dpkt читает и PCAP, и PCAPNG на чистом Python, так что весь инструмент ставится из одного pip и работает офлайн. См. ARCHITECTURE.md, src/capture/.

2. Что если протокол зашифрован?

Это вне области: инструмент сообщает байты как они есть и не делает утверждений об их смысле. Он всё равно перечисляет TCP-сессии и потоки байт, поэтому обёртка видна даже когда нагрузка нет. См. таблицу ограничений в README.md, docs/SUBMISSION.md.

3. Как вы проверяете корректность на новых данных?

Правило применяется без изменений к захвату, под который оно не подгонялось, и каждое сообщение классифицируется. Правило v2 переносится на corpus_capture_02 с 40 matched и 0 контрпримеров и открыто падает на synthetic_live. См. REPORT.md разделы 7–8, tests/integration/test_reference_report.py.

4. Почему 794 теста, а не больше?

Набор покрывает каждый модуль и каждый статус, который движок может испустить, включая граничные и дефектные фикстуры; покрытие — 90 % по src/, а оставшиеся строки — ветви ошибок, для которых нужен настоящий сбой ввода-вывода. Больше тестов не подняло бы утверждение, которое — «подтверждено в области», а не «доказано». См. docs/AUDIT.md разделы 4 и 11. Число — счётчик на архивной сдаче v1.0.0; рабочая main с тех пор выросла до 957.

5. Что происходит при конфликтующем перекрытии?

Пересборщик сохраняет первые байты, записывает диагностику ambiguity с обеими версиями, а поле, покрывающее перекрытие, сообщается как ambiguous, а не matched. Ничего не перезаписывается молча. См. tests/protocol/test_engine.py::test_ambiguity_in_field_range_is_ambiguous.

6. Почему dpkt, а не PyShark?

PyShark вызывает tshark, который должен был бы присутствовать на целевой машине; эталонное окружение этого не гарантирует. У dpkt нет внешнего процесса, и он достаточно быстр для 250k пакетов. См. ARCHITECTURE.md.

7. Что такое провенанс и почему он важен?

Каждый байт пересобранного потока несёт пакет, из которого пришёл: индекс, номер последовательности, временную метку и смещение. Это делает каждый вердикт проверяемым по pcap, а не просто правдоподобным. См. `src/capture/provenance.py`, `docs/GUI_VERIFICATION.md`.

8. Как работает контрпример?

Контрпример — `mismatched`-сообщение, привязанное к своим байтам, смещению и исходному пакету, сохранённое в отчёте, а не отброшенное. Восемьдесят контрпримеров v1 на `corpus_capture_01` — то, что расширило правило до v2. См. `docs/CORPUS_ANALYSIS.md` раздел 4, `docs/RULE_V1_V2.md`.

9. Что вы не сделали и почему?

Нет интерпретации `B_to_A` по полям в поставляемом демо-правиле и нет захвата с живого устройства; каждое заявлено как предел, а не скрыто. Меню проекта в окне покрывает переносной проект, и движок учитывает область `B_to_A`, но демо-правило ограничено только `A_to_B`. Набор организаторов — сгенерированный фрейминг поверх настоящих байтов приложения, поэтому пассивный захват — следующий шаг проверки. См. `REPORT.md` раздел 12, `docs/REAL_CORPUS_ANALYSIS.md` раздел 9.

10. Как вы гарантируете воспроизводимость?

Каждое число берётся из поставляемого файла и названной команды, а окружение закреплено в `pyproject.toml` (Python 3.12+, фиксированные версии зависимостей). Проект хранит относительные пути и контрольные суммы sha256, поэтому снова открывается без изменений на другой машине. См. `docs/PORTABILITY.md`, `docs/CONTRACT.md`.

11. На каких данных вы проверяли?

На двух корпусах: синтетическом (`tests/corpus/`, три захвата, 369 сфреймленных сообщений) и наборе организаторов (`tests/corpus/real/`, четыре исследовательских захвата плюс восемь диагностических, 45 запросов + 45 ответов). См. `docs/CORPUS_ANALYSIS.md`, `docs/REAL_CORPUS_ANALYSIS.md`.

12. Что если файл PCAP повреждён?

Повреждённый, но распознаваемый файл читается насколько возможно; остаток становится диагностиками. Обрезанные заголовки, неподдерживаемые типы канального уровня, неполные фрагменты и смешанные типы канального уровня сообщаются, а не выбрасываются исключением. Фрагментированная датаграмма собирается, когда все её байты присутствуют. См. `docs/METRICS.md` (Robustness), `tests/fixtures/defects/`.

13. Как он масштабируется до 100k сообщений?

Эталонный профиль (250000 пакетов, 1000 сессий, 95.66 МиБ) разбирается, собирается и выгружается за 12.10 с и пикует на 350 МиБ против бюджета 4 ГиБ. Потоковый режим снижает пик до 224 МиБ. См. `docs/BENCHMARK.md`, `docs/METRICS.md`.

14. Что если правило неверно?

Неверное правило сохраняется, а не удаляется: оно остаётся со своей версией, его результат помечается `outdated`, а произведённые им контрпримеры — доказательство для следующей версии. Читатель всегда может сказать, какое правило произвело какой вердикт. См. `src/hypothesis/versioning.py`, `tests/hypothesis/test_versioning.py`.

15. Как перенести исследование на другую машину?

Исследование — переносимый каталог: относительные пути плюс контрольные суммы sha256, экспортируемый и импортируемый как zip с проверкой целостности. Чистая распаковка прогоняет весь набор (793 пройдено, 1 пропущено из опубликованного архива). См. `docs/PORTABILITY.md`, `src/project/`.

Куда смотреть дальше

- Числа с их источником: `docs/METRICS.md`.
- Длинный список вопросов: `docs/DEFENSE_FAQ.md`.
- Предполётный список: `docs/DEFENSE_CHECKLIST.md`.